

Data Analysis in Physical Oceanography

LECTURE 4

SuSe26

Eleanor Frajka-Williams, Office 4B.049 (Bu57)

Eleanor.Frajka@uni-hamburg.de

Data analysis techniques for observational data

Content: Equip you to approach oceanography from an experimental standpoint.

Intended participants: oceanography students

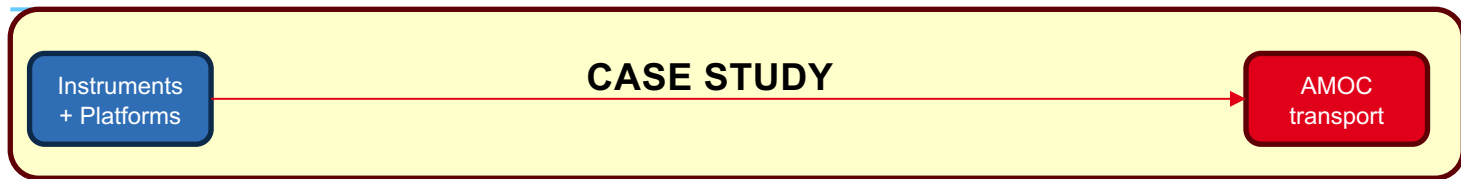
- who will go to sea in the near future
- who will be modellers or theoreticians
- who will become seagoing scientists

Case studies (2026 assignments): Observing the ocean circulation using moored observations

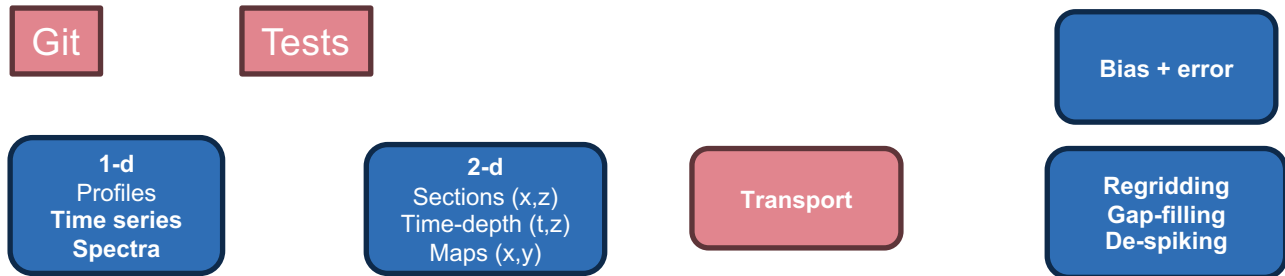
Practical skills:

- Scientific programming using **python** – for robust and reusable code
 - Code structure, modules, functions + **tests**
- **Git** for version control and sharing code
 - Reviewing code, making and approving pull requests, handling issues
- **AI tools** allowed: Claude code, Copilot, ChatGPT, etc. Use them wisely. See “tests” above.

How will we get there? Course plan...



Lab component



Assessments + expected deadlines

Activity	Percentage
Assignment 1: Analysis of AMOC	30%
Assignment 2: Calculating transport from moored arrays	60%
Exercise / participation	10%
Total	100%

- Assignment 1: Due June 19
- Assignment 2: Due Sept 4

Assignments will be submitted as git repositories.

1. Working tests
2. Working demo notebook
3. Modules for functions
4. Short report (figures + text) as report.md

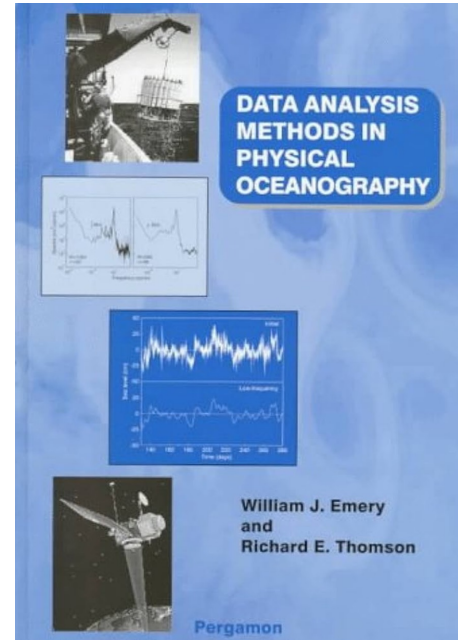
Exercises/Participation will include

- 1 figure discussion session (plan to prepare a figure + give feedback to peers) – deadline June 8
- Peer code feedback – deadline July 3

Textbook

- Emery + Thomson, Data analysis for physical oceanographers (2001, 2014, ..)
- Available at UHH
- Log in with UHH at <https://www.sciencedirect.com/book/9780444507563/data-analysis-methods-in-physical-oceanography>

Multiple chapters!



Course meeting times (Irregular – see Stine)

Mostly Mondays:

- 08:30 – 12:00 in GEOM 433

Preliminary sessions in **April**

- 13 April: Overview of tools
- **Before** April 27 -
 - Get VSCode, python, environment management working on your computer
 - Add your GitHub username to this sheet:
<http://bit.ly/4v9yPWe>
(create an account if you don't have one)
- 27 April: Troubleshooting

1	Mon, 13. Apr. 2026	10:15	11:45	Geom 433
2	Mon, 27. Apr. 2026	10:15	11:45	Geom 433
3	Mon, 18. May 2026	08:30	12:00	Geom 433
4	Mon, 1. Jun. 2026	08:30	12:00	Geom 433
5	Mon, 8. Jun. 2026	08:30	12:00	Geom 433
6	Mon, 6. Jul. 2026	08:30	12:00	Geom 433
7	Mon, 31. Aug. 2026	08:30	12:00	Geom 433

By 18 May, everyone should have all tools installed.

Reviewing each other's code

Peer code review — the code categories (2.0 = the standard taught)

Category (weight)	Exceptional (1.0)	Solid standard (2.0)	Passed (3.0)
Code organization (30%)	clean, logical modules; notebook reads as a narrative	functions in the right modules; notebook imports and uses them	some functions in modules; notebook use is patchy
Code quality (10%)	idiomatic; full docstrings; type hints throughout	passes ruff; basic docstrings; readable	ruff still reports issues; minimal docstrings
Testing & reproducibility (20%)	tests cover your functions + an edge case; runs cleanly	required tests pass; minor setup only	few tests, low coverage; runs only after fixes

Objective checks a reviewer runs: installs & runs from a clean clone; pytest green (+ pytest --cov); ruff check / ruff format clean. Content & writing are graded separately.

Code review assignments (2 reviews each)

This lecture: correlation and trends

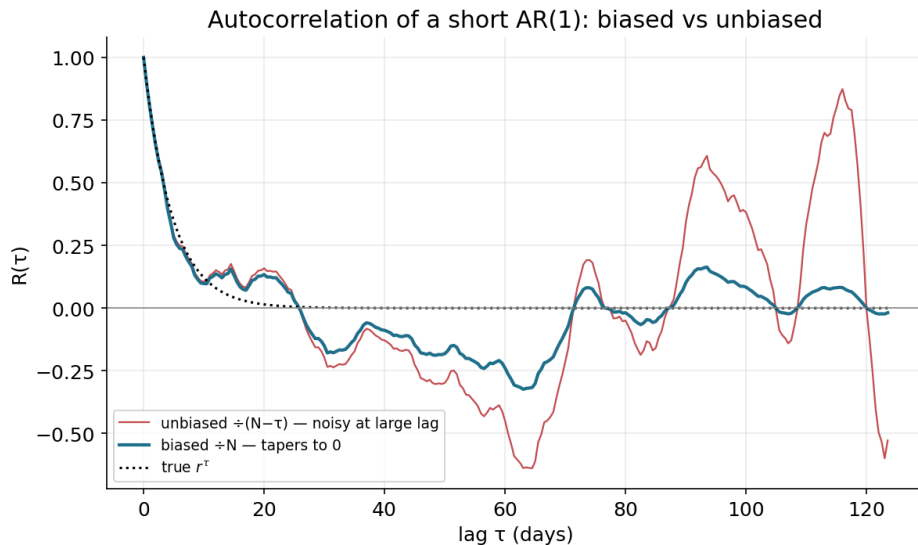
- Correlation is the time-domain twin of last week's spectrum
- Autocorrelation → the decorrelation timescale → how many independent samples you really have
- That number decides whether a trend (or a link between two series) is real or red-noise chance
- Running example: the AMOC and the components you built (Gulf Stream, Ekman, geostrophic upper-mid-ocean)

1 · AUTOCORRELATION AND PERSISTENCE

The autocorrelation function

- $R(\tau) = C(\tau)/C(0)$, the correlation of the series with itself at lag τ ; $R(0) = 1$
- It measures persistence — how long the series 'remembers' itself
- Oceanographic series are red: $R(\tau)$ decays slowly
- Reference cases: white noise $\rightarrow$ a spike at 0; $AR(1) \rightarrow R(\tau) = r^\tau$ (exponential)

Biased vs unbiased estimator

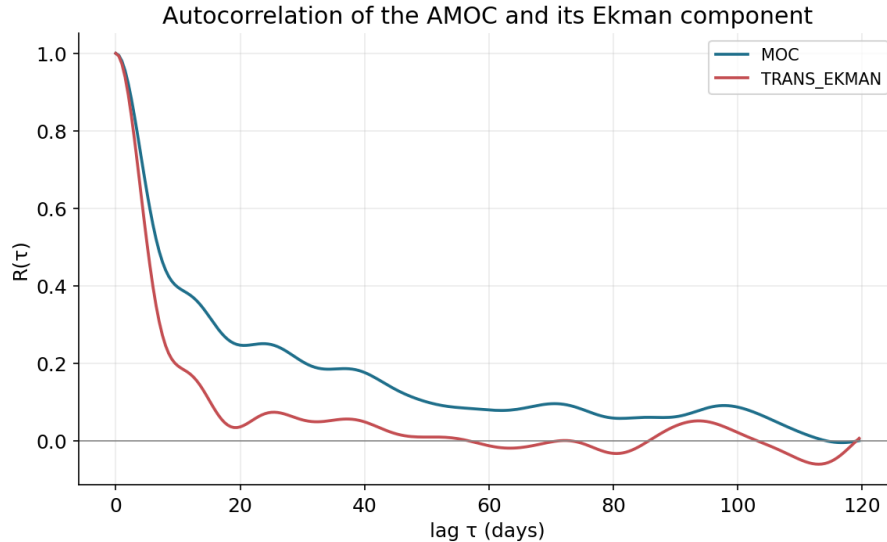


A short AR(1): unbiased $\hat{r}^\tau$ blows up at large lag; biased $\hat{r}_b^\tau$ tapers to 0

Biased vs unbiased — which to use

- Biased ($\div N$): tapers to zero at large lag (triangular weighting), low variance, positive-definite
- Unbiased ($\div (N-\tau)$): unbiased on average but wild at large lag (few pairs)
- Use biased for spectra and integral scales; they agree anyway once you truncate at the first zero

Autocorrelation of the AMOC and Ekman



The MOC decays over weeks; the wind-driven Ekman component forgets itself faster

Code — autocorrelation

```
import numpy as np

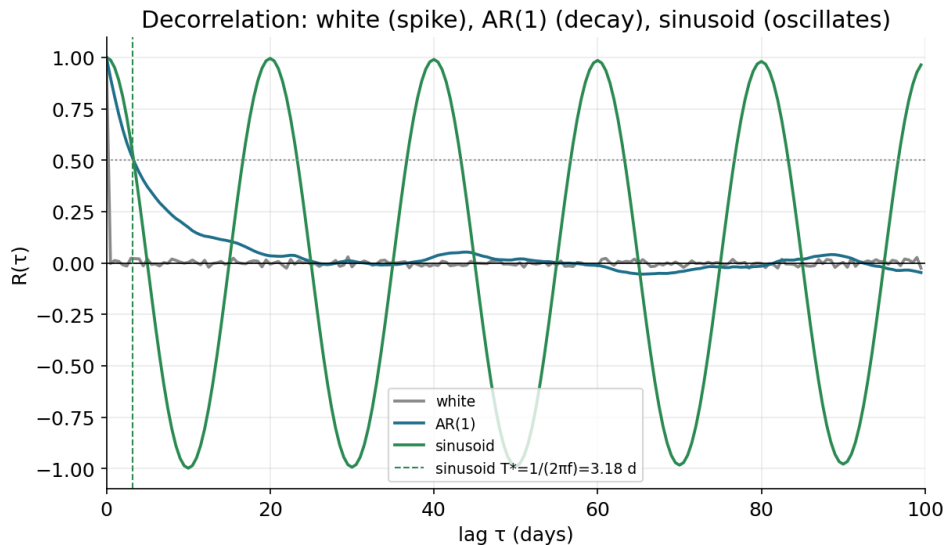
def autocorr(x, biased=True):
    d = x - x.mean()
    n, var = d.size, d.var()
    R = np.correlate(d, d, "full")[n-1:] # lags 0..n-1
    if biased:
        return R / (n * var) #  $\div N$  ( $R(0)=1$ )
    return R / ((n - np.arange(n)) * var) #  $\div (N-\tau)$ 
```

2 · THE DECORRELATION TIMESCALE

The integral timescale T^*

- $T^* = \int R(\tau) d\tau$ — the area under the autocorrelation
- In practice: integrate to the FIRST ZERO CROSSING (past that it is sampling noise)
- Geometrically: T^* is the width of a height-1 box with the same area (equal-area width)
- Detrend first — a trend adds long-range correlation that is not stochastic memory

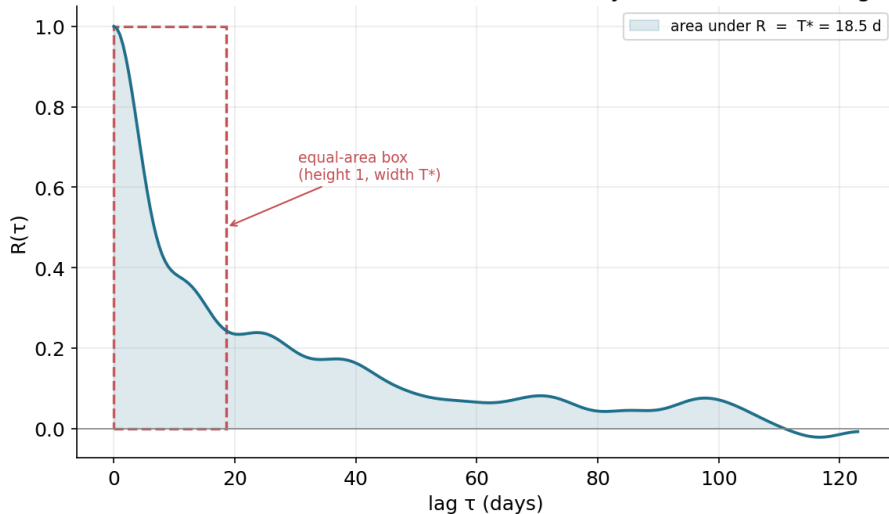
Three cases: white, AR(1), sinusoid



A sinusoid oscillates forever — its integral gives $T^ = 1/(2\pi f)$: periodicity $\neq$ persistence*

The AMOC decorrelation timescale

AMOC decorrelation timescale: $T^* = 18.5$ d (first zero only at 111 d — real long-lag memory)



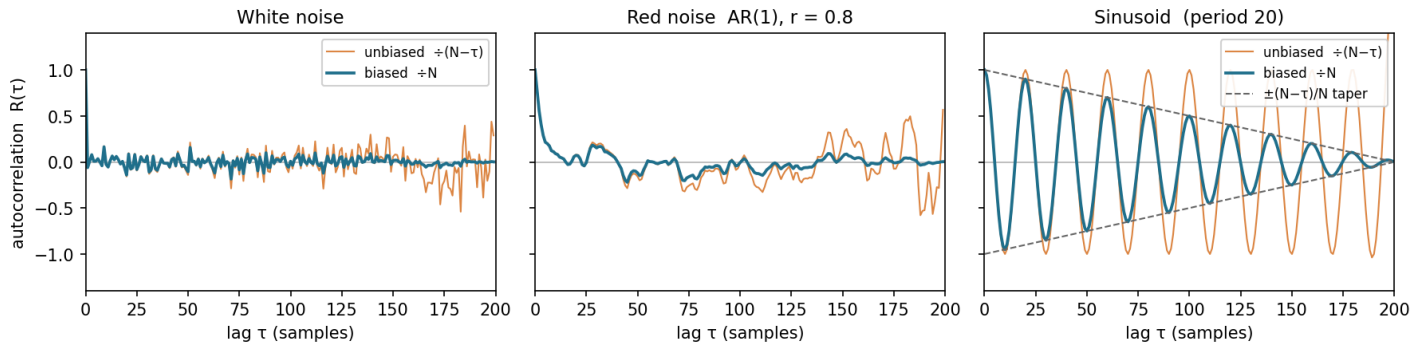
$T^* \approx 18$ d (the shaded area = the dashed box); the first zero is far out — real long-lag memory

Why stop at the first zero crossing?

- The integral timescale is defined as the area to infinity, but the estimated $R(\tau)$ is mostly noise at large lags, so the running sum often never settles.
- Emery & Thomson (§ 5.3 Correlation): sum until it reaches a near-constant plateau — that value is T^* .
- If it never plateaus — our AMOC case, with real interannual variability — integrate only to the first zero crossing (E&T, after Poulain & Niiler, 1989).
- Truncating there trades a tiny bias for a big cut in variance: the noisy tail is thrown away.
- Use the biased ($\div N$) estimator — its tail-damping is what makes the truncated integral stable. The biased-vs-unbiased distinction is subtle but it matters here.

Biased vs unbiased — where the difference shows up

Biased vs unbiased autocorrelation — the difference lives at large lag



Biased = unbiased $\times (N-\tau)/N$ (triangular Bartlett/Fejér taper). At large lag the unbiased estimate fans out where few pairs remain; the biased one is damped toward zero.

Integral timescale → DOF → EDOF (E&T)

Integral timescale → DOF → EDOF (E&T §5.3, §5.8.6.1; EDOF = DOF/2)

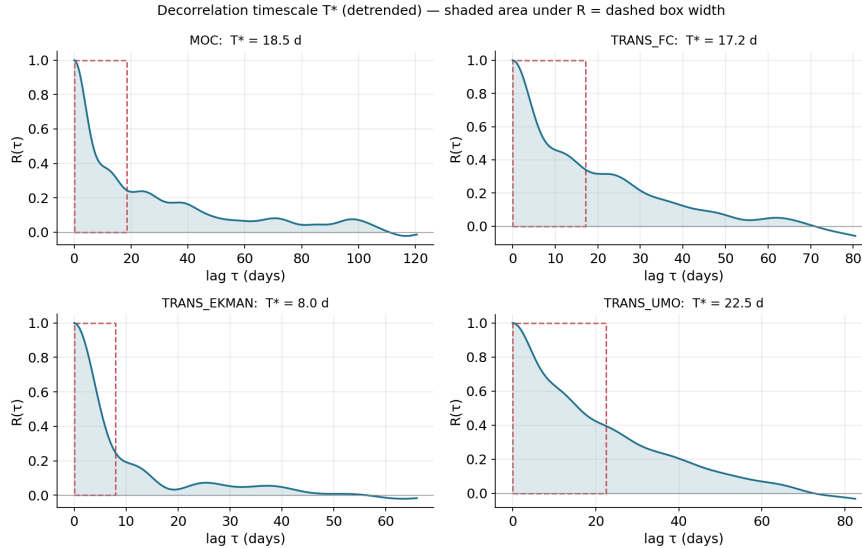
Quantity	How you calculate it	Size	What it's for
Integral timescale T^*	$T^* = \int_0^\infty R(\tau) d\tau$ $\approx (\Delta\tau/2) \sum [R(\tau_i) + R(\tau_{i+1})]$ truncated at the first zero crossing (for the biased R this $\approx$ the full integral)	the timescale	physical MEMORY / decorrelation time
Degrees of freedom DOF	$DOF = \text{record} \div T^* = N \Delta\tau / T^*$	count of decorr. times	rough independent- sample count
Equivalent d.o.f. EDOF	$EDOF = DOF / 2 = \text{record} \div (2 T^*)$	$\frac{1}{2} \times DOF$	ERROR BARS, confidence intervals, significance

EDOF = DOF/2 is what goes into error bars and significance — the factor of 2 is the two-sided variance sum.

Effective degrees of freedom

- The number of INDEPENDENT samples: $N_{\text{eff}} = \text{record} / (2T^*) = N / (1 + 2 \sum \rho_k)$
- The factor of 2: correlation extends on both sides of each point
- AMOC: $T^* \approx 18$ d, so ~20 years of 12-hourly data hold only ~200 independent samples
- This is the number that goes into every error bar and significance test

Decorrelation timescale of the components



Ekman forgets fastest ($T^ \approx 8$ d); the geostrophic upper-mid-ocean is the most persistent ($T^* \approx 24$ d)*

Code — decorrelation timescale & N_eff

```
def integral_timescale(R, dt):          # R from autocorr(), biased
    tau, k = 0.0, 0
    while k+1 < len(R) and R[k] >= 0: # integrate to first zero
        tau += dt*(R[k]+R[k+1])/2; k += 1
    return tau, k                       # one-sided T*, and the crossing index

tau, k = integral_timescale(R, dt)
n_eff = n / (1 + 2*R[1:k].sum())      # effective samples (two-sided!)
```

3 · CROSS-CORRELATION AND THE AMOC COMPONENTS

Relating two series: lead and lag

- Cross-correlation of x and y at lag τ ; the peak lag reveals which leads
- Trap 1: shared trends or seasonality fake a correlation — detrend first
- Trap 2: autocorrelation inflates the significance of a correlation coefficient (use N_{eff})

Remove the seasonal cycle first (xarray groupby)

Two series that share an annual cycle cross-correlate at ± 12 months even when otherwise unrelated, and a seasonal signal can bias a trend. Remove it first.

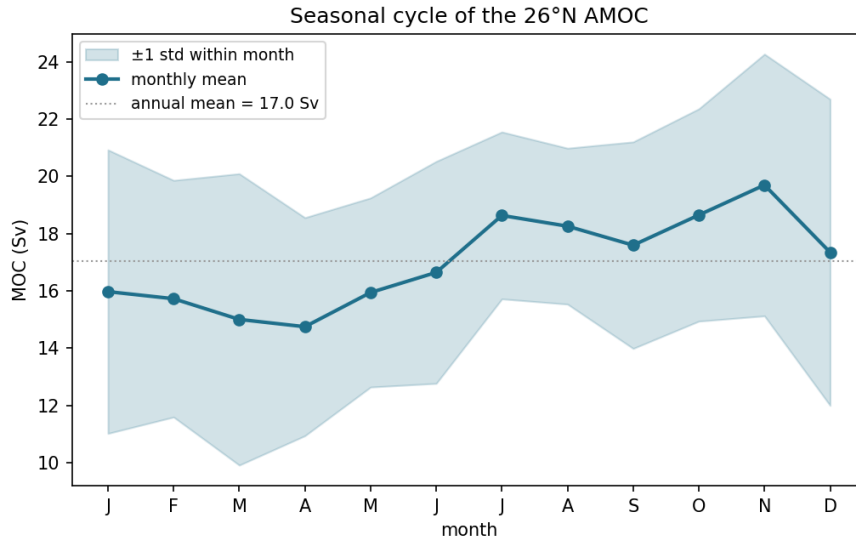
```
import xarray as xr

# monthly climatology = the mean annual cycle (12 values)
clim = da.groupby("TIME.month").mean()

# remove the seasonal cycle but KEEP the overall mean
deseason = da.groupby("TIME.month") - clim + da.mean()

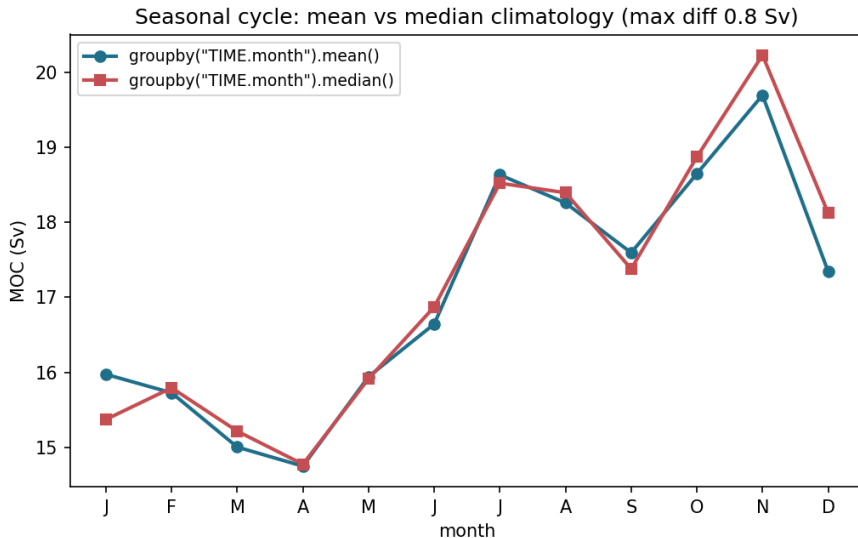
# now correlate / trend the deseasonalised series
```

The seasonal cycle of the 26° N AMOC



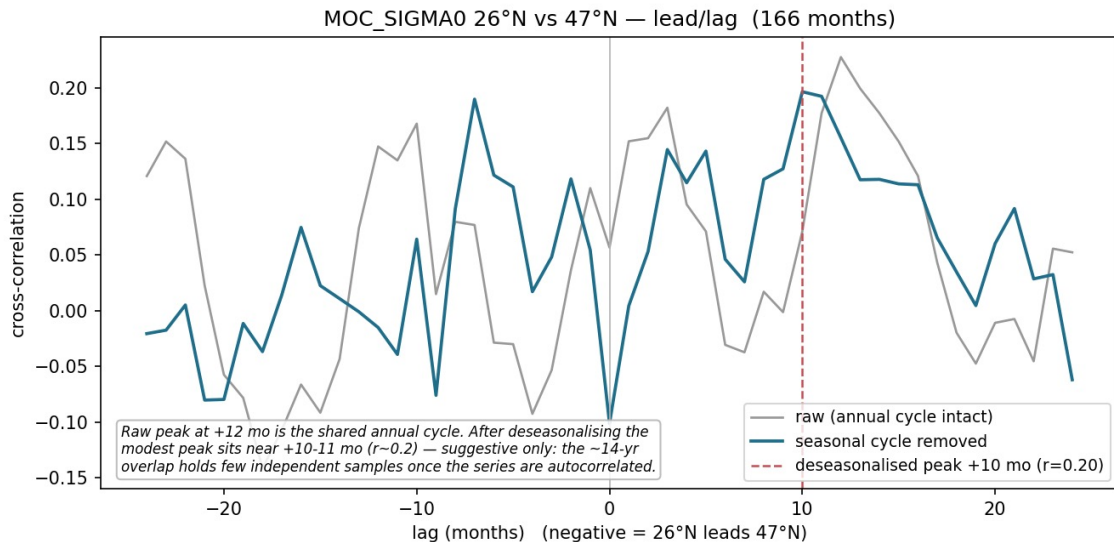
Monthly climatology of the 26° N AMOC: weakest in spring, strongest in autumn; the band is ± 1 std within each month.

Seasonal cycle: mean vs median climatology



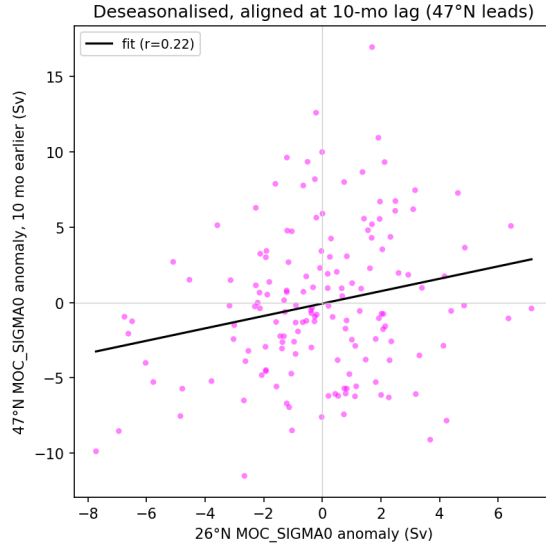
Same shape, max difference ~0.8 Sv. The mean sits below the median in autumn/winter (pulled down by low-transport events); the median is more robust to skew. One-word swap: .mean() -> .median().

Seasonal aliasing in practice: 26° N vs 47° N AMOC



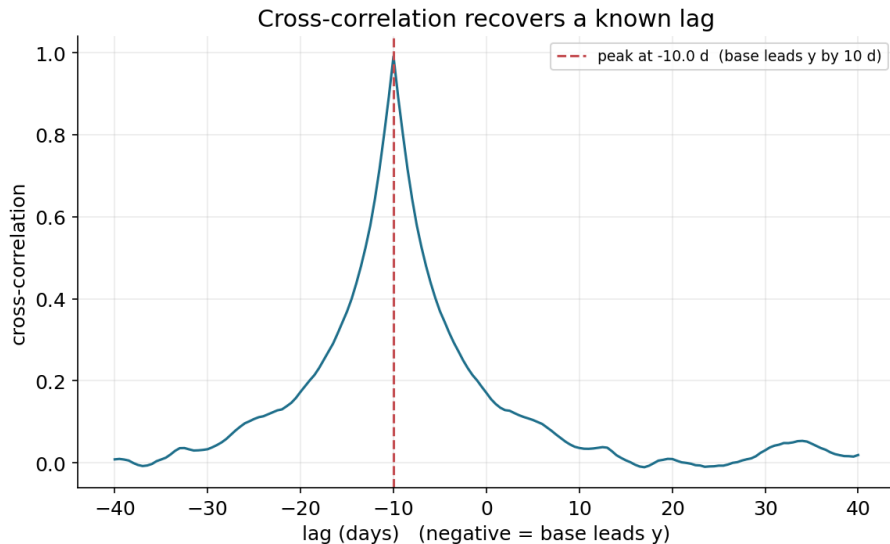
MOC_SIGMA0. Raw cross-correlation peaks at +12 mo (the shared annual cycle); after deseasonalising, a modest ~10-mo lead of 47° N over 26° N remains.

The ~10-month lead as a scatter (deseasonalised)



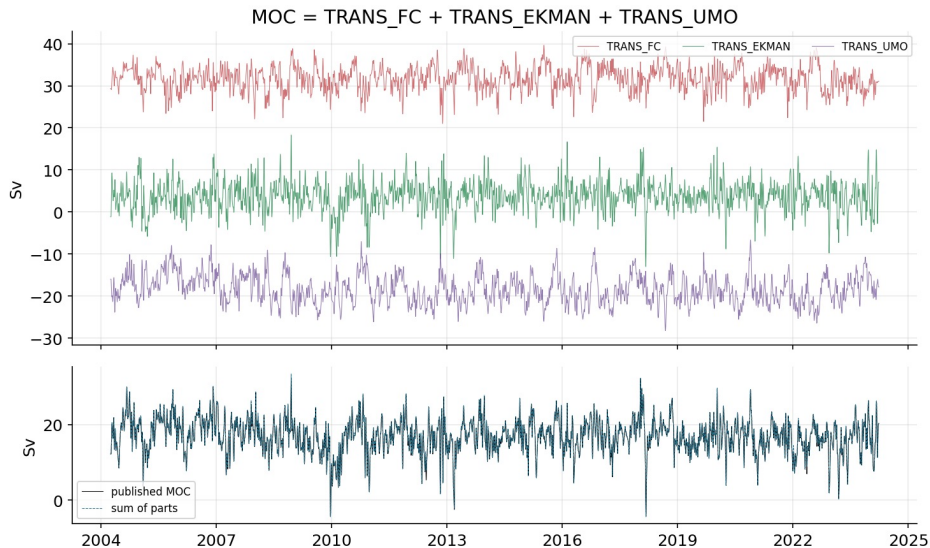
26° N anomaly vs 47° N anomaly 10 months earlier: a weak positive relationship ($r \approx 0.2$) — suggestive, given few independent samples over the 14-yr overlap.

Cross-correlation recovers a known lag



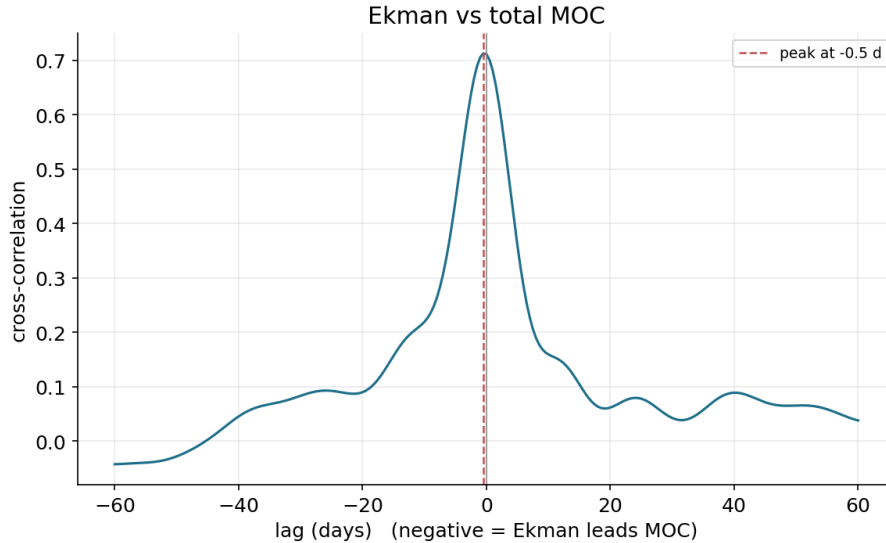
Constructed check: y is a shifted copy of $x \rightarrow$ the peak sits at the planted lag

The AMOC from its parts



$MOC = TRANS_FC + TRANS_EKMAN + TRANS_UMO$ — the sum matches the published MOC to ~ 0.08 Sv

Ekman vs total MOC



Peak near zero lag: Ekman is a direct, contemporaneous component of the overturning

How the geostrophic component relates to the total

- TRANS_UMO — the geostrophic interior transport you built in the geostrophy lecture
- $\text{corr}(\text{TRANS_UMO}, \text{MOC}) \approx 0.28$ — a modest, positive lever on the total
- The overturning is a tug-of-war between a large Gulf Stream and a compensating interior return

Code — cross-correlation

```
from scipy import signal

def cross_corr(x, y):
    x = (x - x.mean())/x.std()
    y = (y - y.mean())/y.std()
    c = signal.correlate(x, y, "full") / len(x)
    lags = signal.correlation_lags(len(x), len(y), "full")
    return lags, c

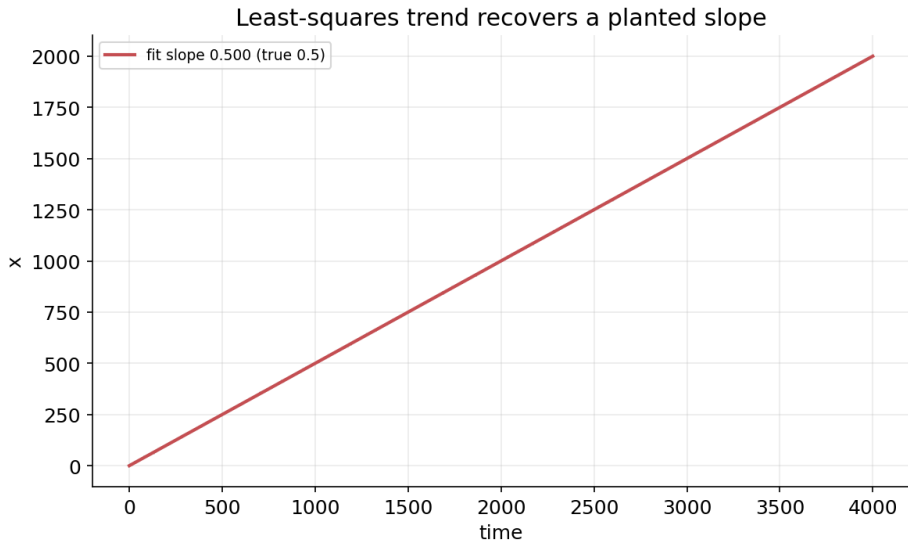
lag = lags[c.argmax()]      # negative => x leads y
```

4 · TRENDS: FITTING AND DETRENDING

Fitting a trend

- A least-squares straight line: the slope is the trend (`numpy.polyfit`)
- Detrend before autocorrelation or spectra — otherwise the trend dominates the low frequencies
- Be wary of higher-order polynomials (overfitting)

Least-squares recovers a planted slope



Constructed check: a known slope buried in noise is recovered within tolerance

Code — trend fit & detrend

```
import numpy as np

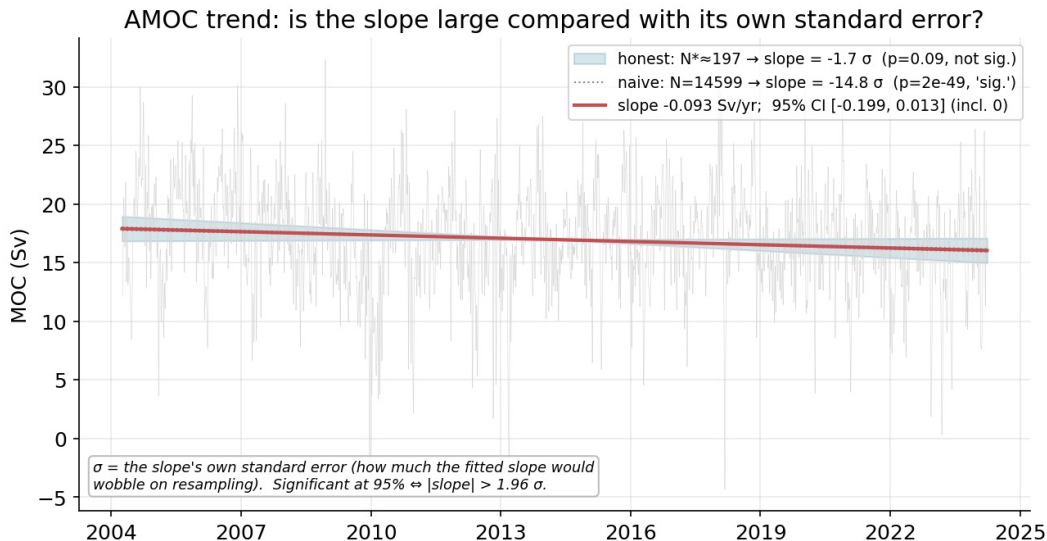
slope, icpt = np.polyfit(t, x, 1)    # t in years -> Sv/yr
trend = slope*t + icpt
resid = x - trend                    # the detrended series
```

5 · IS THE TREND SIGNIFICANT?

Autocorrelation inflates significance

- Naive test: slope / standard error using all N points $\rightarrow$ wildly over-confident
- Fix: use the effective sample size N_{eff} (Part 2) in the standard error and t-test
- The AMOC: naive $p \approx 2 \times 10^{-49}$ collapses to an honest $p \approx 0.09$ — not significant at 95%
- The record is too short and too persistent to call the decline robust

AMOC trend: naive vs honest



σ = the standard error of the slope (its sampling wobble). 95%-significant needs $|\text{slope}| > 1.96\sigma$: naive 14.8σ passes, honest 1.7σ fails — its 95% CI includes zero.

Attribution across the components

- Which part carries the AMOC's variance, and any (honest) trend?
- Put the components side by side with their decorrelation timescales and trends
- Ties the correlation toolkit back to the physics you built

Code — trend significance with effective d.o.f.

```
from scipy import stats

Sxx = np.sum((t - t.mean())**2)
se   = np.sqrt(np.sum(resid**2)/(n-2)/Sxx)    # OLS slope SE
se_eff = se * np.sqrt(n / n_eff)            # inflate for autocorrelation
p = 2*stats.t.sf(abs(slope/se_eff), n_eff-2)
```

Recap: the spine

- Autocorrelation $\rightarrow$ memory (the decorrelation timescale T^*)
- $T^* \rightarrow$ the number of independent samples $N_{\text{eff}} = \text{record}/(2T^*)$
- $N_{\text{eff}} \rightarrow$ honest error bars and trend significance
- Attribution $\rightarrow$ which component drives the AMOC's variability and trend

Format and lint with ruff

ruff formats to one consistent style and flags likely bugs (unused imports/variables, undefined names) — one fast tool, so reviewers read the logic, not whitespace.

```
# before
import numpy as np
import os

def welch( x,y ):
    n=len(x)
    z = 2
    return np.mean(x)+n
```

```
# after: ruff format + ruff check --fix
import numpy as np

def welch(x, y):
    n = len(x)
    return np.mean(x) + n
```

ruff reindented to 4 spaces, spaced the operators, and removed the unused import (os) and unused variable (z).

Test coverage: pytest --cov

Coverage shows which lines your tests actually run — a high % means your tests check the code you wrote, not just import it. Aim to cover the functions you implement.

```
# once:
pip install pytest-cov

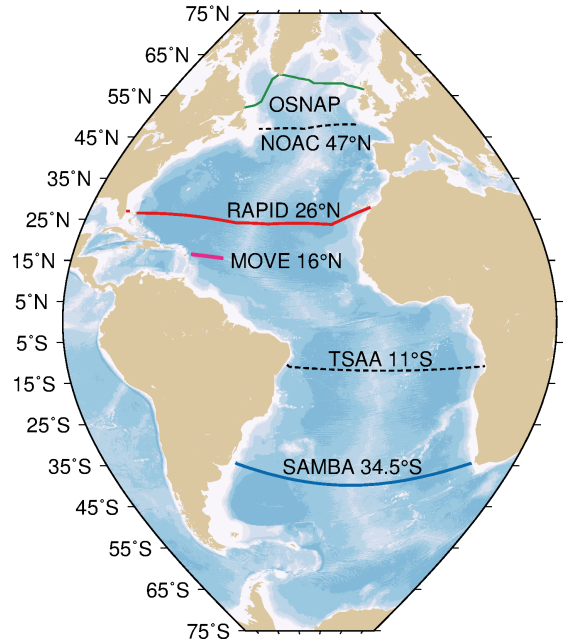
# summary % for your package
pytest --cov=correlation_trends

# list the exact lines no test touches
pytest --cov=correlation_trends --cov-report=term-missing

# click-through HTML report -> htmlcov/index.html
pytest --cov=correlation_trends --cov-report=html
```

See slides in lecture3: geostrophy

CODING TASKS



Task for today + next week

- Today: Plan out how to calculate interior geostrophic transport from the RAPID mooring data.

As part of assignment 2:

1. Find the correct dataset using AMOCatlas
2. Convert to TEOS-10
3. Calculate dynamic height
4. Calculate T_{int} based on McCarthy et al. 2015 equation (1.4)
5. Compare this to the calculated transports from other rapid files, e.g. `moc_transports.nc`
 1. Choose comparison dataset(s)
 2. Integrate the T_{int} in pressure coordinates (native grid) to compare; suggest `trapz` integration
 3. Convert the result to Sverdrups
 4. Verify signs! Compare with FW et al. 2018

Can you explain the differences?

See on Moodle

7 · ASSIGNMENT STRUCTURE

Assignment 2 — Geostrophic transport and relating AMOC series

Course: Data Analysis for Physical Oceanography (MSc)

Weight: Assignment 2 of 2

Issued: 6 July 2026

Due: 4 September 2026 (submitted as a git repository)

Effort: ~3 weeks

Aim

Two things, in two parts. First, **compute** an ocean transport yourself from hydrography using geostrophy (thermal wind), and check it against a published product. Second, **relate time series to one another** — quantify how a single series remembers its past, whether it trends, and whether two series lead, lag, or move together.

The two parts are deliberately independent: **Part 2 uses time series we provide, so you can complete it in full even if your Part 1 geostrophy calculation does not reproduce the published transport well.** A geostrophic estimate that only half works is still a good result to analyse and explain.